Kandypack Project — Progress Summary

Project: Rail and Road-based Supply Chain Distribution System for Kandypack (Kandy, Sri Lanka) **Status**: Database portion fully complete. Backend and frontend not yet started.

1. ER Diagram / Database Design

Designed and iteratively corrected in Lucidchart, covering 14 entities:

CITY — lookup table for the 6 rail-connected cities + Kandy origin
STORE — one per destination city, near the railway station
ROUTE — last-mile delivery routes, linked to a store
TRUCK — delivery vehicles, linked to a store
DRIVER / **ASSISTANT** — independent entities, linked into schedules via dispatch
TRAIN — physical trains
TRAIN_SCHEDULE — individual trips (a train running to a city on a specific date/time), holding its own cargo capacity
CUSTOMER — wholesalers, retailers, end customers
CUSTOMER_ORDER — order header (dates, status, amount)
ORDER_ITEM — line items per order (product + quantity), many-to-many bridge between orders and products
PRODUCT — catalog items with price and space consumption rate
ORDER_TRAVEL — links an order to its assigned route and train trip
TRUCK_DISPATCH — a scheduled truck run: truck + driver + assistant + route + time window

Key corrections made during design review:

Split single-product orders into a proper ORDER/ORDER_ITEM many-to-many structure
Removed incorrect direct driver/assistant → route links (these go through TRUCK_DISPATCH only)
Moved `available_capacity` from TRAIN to TRAIN_SCHEDULE (capacity is per-trip, not per physical train)
Added TRUCK as a proper entity (was previously a free-text field)
Added CITY as a lookup table (was previously free-text city fields scattered across STORE, CUSTOMER, TRAIN_SCHEDULE)
Added full data types to every column

2. Database Implementation (MySQL, via MySQL Workbench)

Database `kandypack` created with all 14 tables, primary keys, and foreign keys
11 CHECK constraints added and individually tested (both pass and fail cases), covering:
7-day minimum order lead time
Order status domain
Positive quantity/non-negative price and amount checks

Logical time ordering (train schedule, truck dispatch)
Non-negative train capacity
Weekly hour upper bounds (driver 40 hrs, assistant 60 hrs)

3. Triggers (9 total, all individually created and tested)

Capacity deduction — deducts a trip's available capacity when an order is assigned, rejecting the insert if it would exceed capacity
Driver no-overlap — blocks overlapping dispatch times for the same driver
Assistant no-overlap — same, for assistants
Truck no-overlap — same, for trucks
Driver no consecutive dispatches — requires a minimum break between a driver's dispatches
Assistant max 2 consecutive routes — blocks a 3rd dispatch within a defined consecutive window
Driver weekly hour cap — blocks a new dispatch if it would push the driver over 40 hrs in that calendar week
Assistant weekly hour cap — same, 60 hrs
Auto-update weekly_hours_logged — keeps DRIVER/ASSISTANT hour totals current after each dispatch

Each trigger was tested with both a passing case and a deliberately violating case, confirming correct rejection behavior. One real bug was found and fixed during testing (a stale test row caused a false negative on the truck no-overlap trigger — resolved by tracing actual table contents rather than assuming).

4. Stored Procedure

`AssignOrderToTrip` — implements the brief's overflow rule: attempts to assign an order to its preferred trip; if capacity is insufficient, automatically searches for and assigns the next available trip to the same city with sufficient capacity and a later departure time. Tested for both the "fits on preferred trip" and "must roll over to next trip" cases.

5. Functions (3 total)

`CalculateOrderSpace(order_id)` — returns total train-space consumption for an order
`CalculateOrderTotal(order_id)` — returns total monetary value of an order
`GetDriverWeeklyHours(driver_id, reference_date)` — returns a driver's total logged hours for the week containing the given date

All three tested against real data and cross-verified against manual calculations.

6. Indexing

Added indexes on foreign key / frequently-queried columns not automatically indexed by MySQL, targeting the 6 required reports:

`ORDER_TRAVEL(route_id)`, `ORDER_TRAVEL(trip_id)`
`CUSTOMER_ORDER(customer_id)`, `CUSTOMER_ORDER(placement_date, delivery_date)`
`ORDER_ITEM(product_id)`

`TRUCK_DISPATCH(driver_id)`, `TRUCK_DISPATCH(assistant_id)`, `TRUCK_DISPATCH(truck_id)`, `TRUCK_DISPATCH(start_time)`
`CUSTOMER(city_id)`, `ROUTE(store_id)`, `STORE(city_id)`

7. Data Population

Generated and loaded via a single SQL script:

7 cities, 6 stores, 10 routes, 15 products, 20 customers
8 drivers, 8 assistants, 8 trucks
3 trains, 12 train trips with defined capacities
40 customer orders, each with 1–3 order items, all satisfying the 7-day lead time rule
40 order-to-trip/route assignments (ORDER_TRAVEL)
15 truck dispatches, spread across dates to avoid roster rule conflicts

8. Reports (all 6 required reports written and verified)

Quarterly sales report (value and volume)
Most ordered items in a given quarter
City-wise and route-wise sales breakdown
Driver and assistant working hours report
Truck usage analysis per month
Customer order history with delivery details

All 6 tested against the live 40-order dataset and returned coherent, correct results.

9. Assumptions Document

A separate document (`kandypack_assumptions.md`) records every assumption made where the brief didn't specify exact details — covering domain interpretations, business rule specifics (break intervals, consecutive-window definitions), data population choices, and UI scope decisions.

What's Left

Area	Status
Database (schema, constraints, triggers, procedures, functions, indexing, data, reports)	✅ Complete
Assumptions documentation	✅ Complete
Backend API	⏳ Not started (planned for Week 2)

Area	Status
Frontend (basic UI)	⏳ Not started (planned for Week 3)
Integration, testing, final docs	⏳ Not started (planned for Week 4)

Full database backup: exported as `kandypack_final.sql` (structure + data + triggers + routines), kept in the project folder as the official recoverable copy of all work completed.